קטע קוד A

מחלקה גנרית שמאותחלת לטיפוס T

```
1.  template <class T>
2.  class Factory
3.  {
4.      static std::set<T*>  available;
5.      std::set<T*>         used;
6.  public:
7.      T* allocate() {
8.          if (available.empty()) {
9.              // see explanation regarding set::insert at the end
10.             return *used.insert(new T()).first;
11.         }
12.         else {
13.             T* t = *available.begin();
14.             available.erase(available.begin());
15.             return *used.insert(t).first;
16.         }
17.     }
18.
19.     void release(T* t) {
20.         available.insert(t);
21.         // set::erase does nothing if 't' is not in the set
22.         used.erase(t);
23.     }
24.
25.     int numUsed() {
26.         return used.size();
27.     }
28.
29.     int numAvailable() {
30.         return available.size();
31.     }
32.
33.     ~Factory() {
34.         for (T* t : used)
35.             delete t;
36.     }
37. };
38. template <class T>
39. set<T*> Factory<T>::available;
40.
41. // explanation regarding set::insert:
42. // set.insert returns std::pair<iterator, bool>
43. // 'first' is an iterator pointing to the new item
44. // or to an item with the same value that already
45. // existed, which prevented the insertion
46. // 'second' is whether insertion was performed
```

להלן מספר main אפשריים:

A1 main

```
int main() {                                   static avl <int> y
    Factory<int> f1;
    int *x = f1.allocate();                    use1 x y z
    int *y = f1.allocate();
    int *z = f1.allocate();
    f1.release(y);
                                               avl <Char> c2
    Factory<char> f2;                          use2 c1 c2
    char *c1 = f2.allocate();
    char *c2 = f2.allocate();
    f2.release(c2);
                                 2                                 1
    cout << f1.numUsed() << ',' << f1.numAvailable() <<
                 1                                 1
     ',' << f2.numUsed() << ',' << f2.numAvailable() << endl;
}
```

A2 main

```
int main() {                                   static int av : y
    Factory<int> f1;
    int *x = f1.allocate();                    use1 x y z
    int *y = f1.allocate();
    int *z = f1.allocate();

    {
        Factory<int> f2;                       use2 y
        f1.release(y);
        y = f2.allocate();
        // f2.release(y);
    }
    //f1.release(y);
                                 2                                 0
    cout << f1.numUsed() << ',' << f1.numAvailable() << endl;
}
```

A3 main

```
int main() {                                   static <int> x
    Factory<int> f1;                           use1: x
    Factory<int> f2;                           use2:
    int *x = f1.allocate();
    f2.release(x);                             static x x
    //f1.release(x); →                         use1:
                                               use2:
    //int *y = f1.allocate()
    //int *y = f2.allocate()

                                 1                                 1
    cout << f1.numUsed() << ',' << f1.numAvailable() <<
                 0                                 1
     ',' << f2.numUsed() << ',' << f2.numAvailable() << endl;
}
```

קטע קוד B

```
class Shape {
public:
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    friend ostream& operator<<(ostream& out, const Shape& s) {
        out << s.area() << ',' << s.perimeter();
        return out;
    }
};
class Polygon : public Shape {
    int ns;
    mutable int *sides;
public:
    Polygon(int numSides) : ns(numSides), sides(nullptr) {}

    int numSides() const { return ns; }
    double perimeter() const {
        if (!sides) {
            sides = new int[ns];
            for (int i = 0; i < ns; ++i)
                sides[i] = sideSize(i);
        }
        double p = 0;
        for (int i = 0; i < ns; ++i)
            p += sides[i];
        return p;
    }
    virtual ~Polygon() {
        delete[] sides;
    }
    friend ostream& operator<<(ostream& out, const Polygon& p) {
        out << p.ns << ',';
        out << (Shape&)p;
        return out;
    }
protected:
    virtual double sideSize(int side) const = 0;
};
class Rectangle : public Polygon {
    int w, h;
public:
    Rectangle(int width, int height) :
        Polygon(4), w(width), h(height) {}
    double area() const { return w * h; }
    double perimeter() const { return 2 * (w + h); }
    friend ostream& operator<<(ostream& out, const Rectangle& r) {
        out << "Rectangle: " << r.w << ',' << r.h << ',';
        out << (Polygon&)r;
        return out;
    }
protected:
    double sideSize(int side) const {
        switch (side) {
            case 0:
```

```
            case 2: return w;
            case 1:
            case 3: return h;
            default: return 0;
        }
    }
};
class Square : public Rectangle {
public:
    Square(int length) : Rectangle(length, length) {}
    friend ostream& operator<<(ostream& out, const Square& s) {
        out << "Square: " << s.sideSize(0) << ',';
        out << (Polygon&)s;
        return out;
    }
};

int main() {
    Rectangle rect(5,5);
    Square square(10);

    cout << Rectangle(square) << endl;
    cout << *dynamic_cast<Rectangle*>(&square) << endl;
}
```

Rectan :rect

w=5
h=5
ns=4
sides=nullptr

square: square

w=10
h=10
ns=4
sides= null ptr

עדיין אובייקט חדש

Rectangle: 10,10,4,100,40

המרה בלי יצירה

קטע קוד C – לשאלות הבונוס

```
1.  template <class T>
2.  class Factory
3.  {
4.      std::set<T*>      available;
5.      std::set<T*>      used;
6.  public:
7.      Factory() = default;
8.      /*Factory(const Factory& f) {
9.          available = f.available;
10.         used = f.used;
11.     }*/
12.     Factory(Factory&& f) {
13.         if (this != &f) {
14.             available = std::move(f.available);
15.             used = std::move(f.used);
16.             // after being moved, an std::set is in a valid
17.             // but unknown state (regarding its items)
18.             f.available.clear();
19.             f.used.clear();
20.         }
21.     }
22.     Factory& operator+=(Factory&& f) {
23.         if (this != &f) {
24.             available.insert(f.available.begin(), f.available.end());
25.             used.insert(f.used.begin(), f.used.end());
26.             f.available.clear();
27.             f.used.clear();
28.         }
29.         return *this;
30.     }
31.     T* allocate() {
32.         if (available.empty()) {
33.             // see explanation regarding set::insert at the end
34.             return *used.insert(new T()).first;
35.         }
36.         else {
37.             T* t = *available.begin();
38.             available.erase(available.begin());
39.             return *used.insert(t).first;
40.         }
41.     }
42.     void release(T* t) {
43.         available.insert(t);
44.         // set::erase does nothing if 't' is not in the set
45.         used.erase(t);
46.     }
47.     int numUsed() { return used.size(); }
48.     int numAvailable() { return available.size(); }
49.     ~Factory() {
50.         for (T* t : used)
51.             delete t;
52.     }
53. };
54.
```

```
// explanation regarding set::insert:
// set.insert returns std::pair<iterator, bool>
// 'first' is an iterator pointing to the new item
// or to an item with the same value that already
// existed, which prevented the insertion
// 'second' is whether insertion was performed

int main() {
    Factory<int> f1;
    f1.allocate();
    f1.allocate();
    f1.release(f1.allocate());
    //Factory<int> f2;
    //Factory<int> f2 = std::move(f1);
    f2.allocate();
    f2.release(f2.allocate());
    //f2 += std::move(f1);

    cout << f1.numUsed() << ',' << f1.numAvailable() << ','
         << f2.numUsed() << ',' << f2.numAvailable() << endl;
}
```

0 0 3 2

בהצלחה !